

Reviewer Pack — Minimal Rank of Tropical Curves Bounds Resolution Proof Width...

Entry 4ff636444ce9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 18:52:20 UTC

Minimal Rank of Tropical Curves Bounds Resolution Proof Width

Entry ID: 4ff636444ce9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 18:52:20 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tropical Curves) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every satisfiable k -CNF F with m clauses on n variables, the resolution proof width $t(F)$ is upper-bounded by a function of the minimal rank of the tropical curve associated with F .’, ‘Specifically, there exists a constant c such that for all satisfiable k -CNFs F , $t(F) \leq c * \text{Rank}(\text{TropicalCurve}(F))$.’, ‘This conjecture implies an exponential lower bound on the resolution proof width in terms of the geometric properties of the associated tropical curve.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework for studying the geometry of polynomials over the max-plus semiring, which can be used to represent boolean functions. By studying the minimal rank of the tropical curve associated with a CNF, we may uncover structural properties that are not apparent in traditional representation.’, ‘If this conjecture holds, it would suggest that the geometric structure of the associated tropical curve plays a significant role in determining the complexity of finding a resolution proof for the CNF.’]

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b88c9fe71b574ace

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that the resolution proof width is bounded by the minimal rank of the tropical curve, a result will be considered supported if, for all k -CNF instances with $n \leq 40$ variables, the ratio of the average resolution proof width to the average minimal rank of the tropical curve over all instances is less than or equal to 1.5 AND no instance has a resolution proof width greater than 1.2 times its corresponding minimal rank.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Constants
MAX_N = 40
MAX_CLAUSES = 150
MAX_VARIABLES = 20
MAX_ITERATIONS = 30

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]

    # Back-substitute
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x
```

```

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    result = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(len(B)):
                result[i][j] += A[i][l] * B[l][j]
    return result

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def grothendieck_witt_class(poly, mod):
    n = len(poly)
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            A[i][j] = (poly[i] + poly[j]) % mod
            A[j][i] = A[i][j]
    det = determinant(A)
    return abs(det) ** (1/n)

def resolution_width(F):
    # Simplified DPLL solver for demonstration purposes
    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clauses = [c for c in clauses if len(c) == 1]
        if unit_clauses:
            literal = next(iter(unit_clauses[0]))
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if dpll([c - {literal} for c in clauses if literal not in c], new_assignment):
                return True
            new_assignment[literal] = False
            if dpll([c - {-literal} for c in clauses if -literal not in c], new_assignment):
                return True
            return False
        pure_literals = [l for l in range(1, len(clauses) + 1) if (all(l in c for c in clauses) or
        if pure_literals:
            literal = pure_literals[0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if dpll([c - {literal} for c in clauses if literal not in c], new_assignment):
                return True
            new_assignment[literal] = False
            if dpll([c - {-literal} for c in clauses if -literal not in c], new_assignment):
                return True

```

```

        return False
    return False

assignment = [False] * (len(F) + 1)
return len(next((assignment for assignment in itertools.product([True, False], repeat=len(F))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    m = random.randint(n // 2, MAX_CLAUSES)
    F = []
    for _ in range(m):
        clause = set(random.sample(range(1, n + 1), random.randint(1, n)))
        F.append(clause)

    tropical_curve_rank = grothendieck_witt_class(F, 2) if m > 0 else 0
    resolution_width_F = resolution_width(F)

    return {
        "metric_name": "resolution_width",
        "metric_value": resolution_width_F,
        "instances_tested": 1,
        "conjecture_holds": resolution_width_F <= 1.5 * tropical_curve_rank and resolution_width_F <= 1.5 * tropical_curve_rank,
        "counterexample": "" if resolution_width_F <= 1.5 * tropical_curve_rank else f"resolution_width_F > 1.5 * tropical_curve_rank"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) # Default seeds

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    avg_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std=0 support_fraction=1")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if r["counterexample"]), "")
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88143ebb.py", line 136, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88143ebb.py", line 119, in run_trial
    tropical_curve_rank = grothendieck_witt_class(F, 2) if m > 0 else 0
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88143ebb.py", line 72, in grothendieck_
    A[i][j] = (poly[i] + poly[j]) % mod
               ^~~~~~
TypeError: unsupported operand type(s) for +: 'set' and 'set'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's validity according to the pre-registered support condition. | next: Investigate and fix the error in the test code that caused it to crash. Once the code is stable, rerun the tests to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11555
2	preregistration	ollama_remote	glm4:latest	0	0	6087
3	novelty	ollama_remote	glm4:latest	0	0	4637
4	novelty	ollama_remote	glm4:latest	0	0	4883
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25433
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14344
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17686
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16749
9	judge	ollama_remote	glm4:latest	0	0	8448

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109821 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/4ff636444ce9.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4ff636444ce9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4ff636444ce9.tar.gz` (if generated)